

# Otimizações de Performance — Guia Técnico

---

Documento de referência para as três frentes de otimização entregues recentemente: gate/relatório de tamanho de bundle no CI, métricas de navegação enviadas ao Sentry sem dependência da lib `web-vitals`, e prefetch antecipado de chunks lazy no hover de listagens. Escrito em tom técnico-didático para onboarding e revisão.

Todas as referências aparecem no formato `caminho/arquivo.ext:linha` para permitir navegação direta em IDEs.

---

## Sumário

---

1. Visão geral e princípios de design
  2. Bundle Size Gate e Report
    - Modelo de dados do baseline
    - Normalização de nome de chunk (hash rotativo)
    - Camadas do gate bloqueante
    - Relatório informativo no PR
    - Workflow e idempotência do comentário
    - Separação gate × report
    - Interpretando o report — guia rápido
  3. Métricas de navegação (Sentry) sem web-vitals
    - Métricas coletadas
    - APIs nativas usadas
    - Normalização de rotas
    - Buffer, flush e sample rate
    - Kill switch em três camadas
    - Integração com o router
  4. Prefetch on-hover em cards
    - API do hook
    - Guardas de execução
    - Comportamento por tipo de evento
    - Padrão de wiring por linha
    - Prefetch do chunk lazy
  5. Apêndice
-

# Visão geral e princípios de design

Três frentes independentes, todas ativáveis/desativáveis sem redeploy e todas com custo runtime desprezível:

| Frente                | Objetivo                                                       | Onde vive                                                                 |
|-----------------------|----------------------------------------------------------------|---------------------------------------------------------------------------|
| Bundle gate/report    | Impedir regressão de peso do JS + dar visibilidade no PR       | CI ( <code>.github/workflows/*.yaml</code> ) + <code>scripts/*.mjs</code> |
| Métricas de navegação | Medir experiência real (TTFB, CLS, TTI, troca de rota)         | <code>src/lib/telemetry/navigationMetrics.ts</code>                       |
| Prefetch on-hover     | Antecipar o chunk lazy da rota-alvo em interações intencionais | <code>src/hooks/common/usePrefetchOnHover.ts</code>                       |

## Restrições que guiaram as decisões

Documentadas para evitar retrabalho futuro:

- **Sem `web-vitals`** — a biblioteca oficial é excelente, mas adiciona peso ao main chunk. As APIs nativas ( `PerformanceObserver` , `PerformanceNavigationTiming` , `Layout Shift API` ) cobrem o que precisamos com zero custo extra no bundle.
- **Sem `service worker client-side`** — decisão explícita para não introduzir complexidade de cache offline / invalidação em uma app fortemente autenticada.
- **Sem redesign de UI** — todas as otimizações são transparentes ao usuário final. `Prefetch` não altera visual, métricas não têm dashboard interno.
- **Kill switch obrigatório em telemetria** — qualquer coleta enviada ao Sentry precisa poder ser desligada tanto por `env var` (build-time) quanto por `localStorage` (runtime, por navegador).

## Bundle Size Gate e Report

Dois scripts complementam-se para vigiar o peso do bundle:

- `scripts/check-bundle-size.mjs:1` — **gate bloqueante** rodado no CI. Enforça limites e falha o build em regressões.
- `scripts/bundle-size-report.mjs:1` — **relatório informativo** postado como comentário do bot no PR. Nunca falha o build.

O snapshot canônico compartilhado por ambos é `bundle-size-baseline.json` .

### Modelo de dados do baseline

O arquivo `bundle-size-baseline.json` tem três blocos:

```
{
  "limits": {
    // ← usado pelo gate
    "maxChunkBytes": 1039733, // teto absoluto por chunk (bytes crus)
    "maxTotalBytes": 13707213, // teto absoluto do total JS
    "warningThresholdPct": 75, // avisa quando chunk ≥ 75% do teto
    "regressionThresholdPct": 15 // falha se chunk crítico cresce >15%
  },
  "criticalChunks": {
    // ← rastreados individualmente
    "react-vendor": { "maxBytes": 225, "label": "React + ReactDOM", "currentBytes": 187 },
    "supabase-vendor": { "maxBytes": 244419, "label": "Supabase SDK", "currentBytes": 203682 },
    // ...
  },
  "snapshot": {
    // ← fonte da comparação vs "main"
    "totalBytes": 11422678,
    "chunkCount": 320,
    "chunksByPrefix": { "AppShell": 45231, "react-vendor": 187, /* ... */ },
    "topChunks": [ /* nome + size dos 10 maiores */ ]
  }
}
```

## Referências:

- Estrutura completa produzida por `--update-baseline` em `scripts/check-bundle-size.mjs:128-147`.
- Consumido pelo gate em `scripts/check-bundle-size.mjs:163-172`.
- Consumido pelo report em `scripts/bundle-size-report.mjs:50-51` e `scripts/bundle-size-report.mjs:96`.

O comando para regenerar após refactor intencional está exposto no `package.json:49`:

```
npm run check:bundle-size:update
```

## Normalização de nome de chunk (hash rotativo)

Vite gera nomes com hash de conteúdo ( `AppShell-BY0BJGWD.js` ). Para comparar entre builds sucessivos precisamos remover o hash. Isso é feito por `extractChunkPrefix()` em ambos os scripts.

Implementação em `scripts/check-bundle-size.mjs:93-100`:

```
function extractChunkPrefix(filename) {
  const withoutExt = filename.replace(/\.js$/, "");
  // Regex greedy `.+` preserva hífen no meio do nome:
  // "react-vendor-Abc12345" → "react-vendor" (correto)
  // Se usássemos `.+?` (lazy), quebraria em "react".
  const match = withoutExt.match(/^(.+)-[A-Za-z0-9_-]{8,}$/);
  return match ? match[1] : withoutExt;
}
```

O mesmo algoritmo aparece em `scripts/bundle-size-report.mjs:36-40` — os dois scripts precisam concordar prefixo a prefixo para que o baseline sirva a ambos.

Chunks sem hash reconhecível (por exemplo `manifest.js`) ficam com o próprio nome como prefixo — comportamento de fallback intencional.

## Camadas do gate bloqueante

O gate roda no job **Gate 3.5** do `quality-gate.yml:100` e enforça quatro camadas encadeadas — a primeira violação em qualquer camada zera o build.

1. **Limite global por chunk** — `scripts/check-bundle-size.mjs:189-199` . Qualquer chunk acima de `maxChunkBytes` é violação; entre 75% e 100% do limite gera warning.
2. **Limite total** — `scripts/check-bundle-size.mjs:202-206` . Soma bruta de todos os `.js` do `dist/assets/` .
3. **Chunks críticos** — `scripts/check-bundle-size.mjs:210-243` . Cada prefixo em `criticalChunks` tem seu próprio `maxBytes` (mais apertado que o global) para que regressões em vendedores específicos vazem antes de contaminar o total.
4. **Regressão vs snapshot** — dentro do mesmo loop, `check-bundle-size.mjs:231-242` . Se um chunk crítico cresceu mais que `regressionThresholdPct` (default 15%) em relação ao `snapshot.chunksByPrefix` , é violação; entre metade do threshold e ele é warning.

O gate imprime também um top-5 dos maiores chunks com deltas ( `check-bundle-size.mjs:268-275` ) para diagnóstico rápido em logs de CI.

## Relatório informativo no PR

`scripts/bundle-size-report.mjs:49-130` gera um markdown com duas tabelas:

- **Chunks críticos** — sempre listados, mesmo quando o delta é zero. Serve como radar visual do que foi tocado.
- **Outros chunks com variação relevante** — filtrado por  $\Delta \geq 20$  KB **ou**  $\Delta \geq 10\%$  ( `bundle-size-report.mjs:81` ), ordenado por delta absoluto e limitado aos 15 maiores ( `bundle-size-report.mjs:120` ).

Os thresholds visuais são alinhados com o gate bloqueante em `scripts/bundle-size-report.mjs:26-46` :

| Símbolo     | Faixa                      | Significado                       |
|-------------|----------------------------|-----------------------------------|
| <b>OK</b>   | $ \Delta  \leq 5\%$        | Ruído de build                    |
| <b>WARN</b> | $5\% <  \Delta  \leq 15\%$ | Merece revisão                    |
| <b>FAIL</b> | $ \Delta  > 15\%$          | Casaria com falha do gate         |
| <b>NOVO</b> | <code>prev = 0</code>      | Chunk novo (baseline não conhece) |

O markdown termina obrigatoriamente com o marcador HTML `<!-- bundle-size-report -->` em `scripts/bundle-size-report.mjs:128` — é essa marca que garante idempotência no comentário do PR (próxima seção).

## Workflow e idempotência do comentário

O workflow `.github/workflows/bundle-size-report.yml:1` roda em `pull_request` , mas com dois filtros:

- **Paths** ( `bundle-size-report.yml:6-11` ) — só dispara quando um arquivo relevante muda ( `src/**` , `package.json` , `package-lock.json` , `vite.config.ts` , `bundle-size-baseline.json` ).
- **Concurrency** ( `bundle-size-report.yml:13-15` ) — cada push cancela o run anterior para o mesmo PR, evitando corrida entre dois relatórios.

O step de upsert do comentário ( `bundle-size-report.yml:50-110` ) é o núcleo da idempotência. Ele resolve quatro problemas históricos:

1. **PRs longos** — `github.paginate(github.rest.issues.listComments, ...)` varre todas as páginas de comentários em vez de parar em 100.

2. **Falso positivo por citação** — o filtro `isSelf` exige simultaneamente `user.login === 'github-actions[bot]'` (ou `user.type === 'Bot'`) e presença do marcador. Um humano citando `<!-- bundle-size-report -->` em outro comentário não é confundido com o bot.
3. **Duplicatas históricas** — se por bug antigo o PR já tinha 2+ comentários com o marcador, o step ordena por `id` (ordem cronológica), atualiza o mais antigo e chama `deleteComment` para os demais. Convergência monotônica.
4. **PRs de fork** — `if: github.event.pull_request.head.repo.full_name == github.repository` pula silenciosamente forks (que não têm `pull-requests: write`) em vez de deixar o step em vermelho.

## Separação gate × report

Regra: **o report nunca falha o build**. Documentada em `scripts/bundle-size-report.mjs:6`:

NÃO falha o build (gate bloqueante fica em `scripts/check-bundle-size.mjs`).

Por quê a separação:

- O gate roda no pipeline crítico (`quality-gate.yml`) com `set -e` implícito.
- O report roda em workflow separado sem `continue-on-error` porque não precisa — o script nunca sai com código  $\neq 0$  quando o baseline existe.
- Se o report falhasse por falta de baseline, um push em uma branch nova bloquearia PRs. Ele emite "sem baseline" e segue (`bundle-size-report.mjs:102-104`).

Também alimenta o `$GITHUB_STEP_SUMMARY` (`bundle-size-report.mjs:160-166`) — o mesmo markdown aparece no resumo do run, útil quando o PR-comment não é útil (workflows em branches).

## Interpretando o report — guia rápido

Quando o comentário do bot aparece no PR, leia nesta ordem:

**1. Cabeçalho Total JS** — o número mais importante. Se o delta absoluto do total já está em `+50 KB` ou mais, algo grande entrou; investigue antes de olhar chunk-a-chunk. Se está `±10 KB`, é ruído normal de rebuild.

**2. Tabela de chunks críticos** — sempre listada. Foque na coluna `Δ %` e no símbolo de status:

| Status      | Faixa                      | O que significa                                   | O que fazer                                                                             |
|-------------|----------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>OK</b>   | $ \Delta  \leq 5\%$        | Ruído de build (minificação, ordem de import)     | Nada. Merge normal.                                                                     |
| <b>WARN</b> | $5\% <  \Delta  \leq 15\%$ | Regressão pequena — merece revisão                | Ver "Ações em WARN" abaixo.                                                             |
| <b>FAIL</b> | $ \Delta  > 15\%$          | Regressão grande — mesma faixa do gate bloqueante | Ver "Ações em FAIL" abaixo. Provavelmente o CI já falhou no <code>quality-gate</code> . |
| <b>NOVO</b> | Sem baseline               | Chunk novo                                        | Confira se o splitting foi intencional.                                                 |

**3. Tabela "Outros chunks com variação relevante"** — só aparece quando há não-críticos com  $\Delta \geq 20 \text{ KB}$  ou  $\geq 10\%$ . Serve para pegar bibliotecas importadas indiretamente que estouraram um

chunk feature-specific.

## Ações em WARN (WARN, 5-15%)

1. **Cheque o `dist/stats.html`** do PR (baixe o artifact do build ou rode `npm run build local`): abra o treemap e procure o chunk crítico destacado. Um bloco novo grande dentro dele indica a lib culpada.
2. **Rode `git diff origin/main -- package.json`** — nova dep pesada? Frequentemente warns em `ui-vendor` ou `icons-vendor` vêm de novos componentes Radix ou ícones importados sem tree-shaking.
3. **Padrões comuns de fix:**
  - Import específico em vez de barrel: `import { Foo } from 'lib/foo'` em vez de `import { Foo } from 'lib'`.
  - `lazyWithRetry` na rota que trouxe o peso, movendo-o para fora do main chunk.
  - Substituir dep pesada por implementação leve (documentar decisão em `mem://`).
4. **Se a mudança é intencional** (ex.: nova feature grande com justificativa), avance para merge — o gate ainda passa. Se for repetitivo, considere atualizar baseline após o merge (próximo item).

## Ações em FAIL (FAIL, > 15%) — o gate bloqueou o PR

O `quality-gate.yml` (Gate 3.5) já falhou. Não force merge sem passar por uma destas rotas:

1. **Regressão não-intencional** — reverta o commit que trouxe o peso, otimize (mesmos padrões da seção WARN) e reabra o PR.
2. **Refactor intencional de chunking** — se você mudou deliberadamente a estratégia de split (ex.: quebrou um vendor em dois, unificou dois chunks feature-specific), atualize o baseline **no mesmo PR**:

```
npm run build
npm run check:bundle-size:update
git add bundle-size-baseline.json
git commit -m "chore(bundle): atualiza baseline após refactor de chunking"
```

Deixe claro no corpo do PR **por quê** o baseline mudou. O comando grava `currentBytes` reais + limites com margem de +20% para crescimento orgânico ( `check-bundle-size.mjs:123` ).

3. **Feature grande com aval** — mesma coisa: atualize o baseline no PR com justificativa no commit. Prefira dividir em PRs menores quando possível para facilitar bisect de futuras regressões.

## Chunks "NOVO" (novos)

Aparecem quando um prefixo não existe no baseline. Duas causas comuns:

- **Splitting mudou** — Vite renomeou/dividiu chunks. Confirme visualmente no `stats.html` e atualize baseline se intencional.
- **Nova feature** — chunk lazy nasceu (ex.: nova rota com `lazyWithRetry` ). Merge normal; o próximo build já registra o baseline.

Nunca aceite **NOVO** sem checar — pode ser sintoma de um `React.lazy` que deveria estar em um chunk existente e foi para o próprio.

## Métricas de navegação (Sentry) sem web-vitals

Um único arquivo concentra toda a instrumentação: `src/lib/telemetry/navigationMetrics.ts:1` . É importado em `src/main.tsx:7` e inicializado após o Sentry em `src/main.tsx:18-19` .

### Métricas coletadas

Definidas no union `MetricName` em `navigationMetrics.ts:29-35` :

| Métrica                      | Fonte                                                             | Semântica                                 |
|------------------------------|-------------------------------------------------------------------|-------------------------------------------|
| <code>ttfb</code>            | <code>PerformanceNavigationTiming.responseStart</code>            | Tempo até o primeiro byte                 |
| <code>dom_interactive</code> | <code>navigation.domInteractive - startTime</code>                | HTML parseado o suficiente para interação |
| <code>dom_complete</code>    | <code>navigation.domComplete - startTime</code>                   | Todos os subrecursos carregados           |
| <code>cls</code>             | Soma de <code>layout-shift</code> sem <code>hadRecentInput</code> | Deslocamento visual acumulado             |
| <code>tti_approx</code>      | Última <code>longtask</code> + janela ociosa de 5s                | TTI aproximado (heurística)               |
| <code>route_change</code>    | <code>performance.now()</code> no rAF pós-commit                  | Duração percebida da troca de rota SPA    |

O TTI aproximado ( `navigationMetrics.ts:169-199` ) merece atenção: em vez de implementar o algoritmo canônico (que exige rastrear rede + `longtasks` + fila de tarefas), a heurística é "quando estivermos 5s sem nenhuma `longtask`, a página está utilizável". O valor reportado é `max(lastLongtaskAt - startTime, domInteractive)` — sempre  $\geq$  `dom_interactive` . Safari sem suporte a `longtask` cai no `try/catch` sem quebrar ( `navigationMetrics.ts:196-198` ).

O CLS ( `navigationMetrics.ts:143-167` ) é acumulado durante toda a sessão e emitido em dois momentos: `visibilitychange`  $\rightarrow$  `hidden` e `pagehide` — cobre tanto `tab-switch` quanto fechamento real da aba. Após emitir, o acumulador é zerado ( `navigationMetrics.ts:157` ), permitindo múltiplas emissões durante uma sessão longa.

O `route_change` ( `navigationMetrics.ts:205-221` ) é medido com duplo `requestAnimationFrame` . O primeiro rAF garante que o React já commitou o novo tree; o segundo garante que o browser já pintou. A diferença entre `performance.now()` nos dois pontos é a duração perceptível para o usuário.

### APIs nativas usadas

Todas de `window.performance` :

- `performance.getEntriesByType('navigation')` — array com `PerformanceNavigationTiming` da navegação inicial ( `navigationMetrics.ts:135-141` ).
- `new PerformanceObserver(cb)` com `{ type: 'layout-shift', buffered: true }` ( `navigationMetrics.ts:146-152` ) e `{ type: 'longtask', buffered: true }` ( `navigationMetrics.ts:188-194` ). O `buffered: true` é crucial: captura entries emitidas **antes** do observer registrar, cobrindo o gap entre o primeiro paint e o `load` event.

- `performance.now()` para medições de alta resolução.

Cada bloco de instrumentação está isolado em `try/catch` ( `navigationMetrics.ts:145` , `:164` , `:187` , `:196` ) — UAs sem suporte à API específica simplesmente não emitem aquela métrica.

## Normalização de rotas

Para que o Sentry consiga agrupar métricas por rota (não por URL individual), `normalizeRoute()` em `navigationMetrics.ts:72-77` substitui:

- UUIDs canônicos → `:id`
- Sequências numéricas de 2+ dígitos → `:id`
- Tokens alfanuméricos  $\geq 16$  chars → `:id`

Assim `/clientes/9c8f2a10-1234-5678-9abc-def012345678` e `/clientes/12345` viram ambos `/clientes/:id` na tag `route` do evento, permitindo agregação por página no dashboard.

## Buffer, flush e sample rate

Dois problemas resolvidos aqui:

**Problema 1** — métricas podem ser emitidas antes de `initSentry()` terminar (o Sentry SDK é lazy-loaded em algumas configurações).

Solução: fila em memória com máximo de 40 eventos ( `navigationMetrics.ts:45-46` ). `emit()` empurra no fim e chama `flushBuffer()` , que dreno tudo com `try/catch` — se `captureMessage` ainda não estiver pronto, o próximo flush resolve ( `navigationMetrics.ts:105-121` ).

O limite de 40 é uma proteção contra vazamento de memória em edge cases (SDK nunca inicializando): o buffer é FIFO por `BUFFER.shift()` ( `navigationMetrics.ts:100` ), então o mais antigo cai fora.

**Problema 2** — enviar 100% das navegações de todos os usuários faria a cota do Sentry estourar em minutos.

Solução: `sampleRate()` em `navigationMetrics.ts:65-69` lê `VITE_NAV_METRICS_SAMPLE_RATE` (default `0.1 = 10%`) e valida a faixa `[0,1]` . O gate probabilístico está em `emit()` ( `navigationMetrics.ts:99` ):

```
if (Math.random() > sampleRate()) return;
```

Simples, sem estado — cada métrica é sorteada independentemente.

## Kill switch em três camadas

Ordem de precedência em `isEnabled()` ( `navigationMetrics.ts:52-63` ):

1. `localStorage.nav_metrics_disabled === '1'` — vence tudo. Kill switch por navegador, útil para debugar sem poluir Sentry local ou para suporte pedir a um usuário específico para desligar.
2. `VITE_ENABLE_NAV_METRICS === 'false'` — desliga globalmente em build.
3. `VITE_ENABLE_NAV_METRICS === 'true'` — liga globalmente em build.
4. **Fallback:** `!import.meta.env.DEV` — ligado em produção, desligado em dev.

A ordem importa: mesmo que a flag esteja `true` em produção ( `.env.production:11` ), qualquer usuário pode fazer `localStorage.setItem('nav_metrics_disabled','1')` no DevTools e parar de enviar métricas até



revogar. Testado em `src/lib/telemetry/__tests__/navigationMetrics.test.ts:34-49`.

## Integração com o router

Toda a mágica de troca de rota depende de um único ponto: `RouteScrollReset` em `src/components/common/RouteScrollReset.tsx:31`.

```
useEffect(() => {
  notifyRouteChange(pathname); // ← instrumentação
  forceReleaseScrollLock();    // ← bugfix Radix
  // ...
}, [pathname, hash, navType]);
```

`notifyRouteChange()` (`navigationMetrics.ts:205-221`) tem duas guardas:

- **Não instrumentada antes do init** — `if (!started) return` (`navigationMetrics.ts:206`). Se `initNavigationMetrics()` não rodou, a função é no-op.
- **Descarta a primeira renderização** — a variável módulo-scope `lastPath` começa `null`, e a primeira chamada apenas registra `lastPath = pathname` e retorna (`navigationMetrics.ts:208-210`). A primeira "troca" na verdade é o mount inicial, cuja duração já é coberta por `ttfb / dom_interactive`.

Para testes, `__resetForTests()` em `navigationMetrics.ts:243-251` limpa buffer, flags e timers. **Não** importar em runtime.

---

## Prefetch on-hover em cards

Ideia: quando o usuário passa o mouse (ou foca ou toca) em um card de lista, o chunk lazy da rota de destino é buscado em background. Quando ele clica, o chunk já está no cache do bundler e a transição é instantânea.

O ponto único de verdade é `src/hooks/common/usePrefetchOnHover.ts:37`.

### API do hook

Assinatura em `usePrefetchOnHover.ts:37-40`:

```
export function usePrefetchOnHover(
  fn: () => void | Promise<unknown>,
  { debounceMs = 120, enabled = true }: { debounceMs?: number; enabled?: boolean } = {},
): PrefetchHandlers;
```

Retorna um objeto (`PrefetchHandlers` em `usePrefetchOnHover.ts:29-35`) que deve ser espalhado no elemento-alvo:

```
const handlers = usePrefetchOnHover(() => import('@pages/foo/Detail'));
return <div {...handlers}>...</div>;
```

Handlers retornados:

- `onMouseEnter`, `onFocus` — agendam com debounce.
- `onMouseLeave`, `onBlur` — cancelam o debounce se ainda não disparou.

- `onTouchStart` — dispara imediatamente (sem debounce).

## Guardas de execução

Três guardas encadeadas em `schedule()` ( `usePrefetchOnHover.ts:51-63` ):

1. **enabled** — permite desligar seletivamente (por exemplo, quando um filtro stale está ativo em `QuotesConfigurableList` ).
2. **firedRef.current** — flag booleano por instância que garante execução única. Depois que `fn()` foi chamado uma vez, todos os eventos futuros são no-op — importante em listas grandes onde o usuário passa o mouse várias vezes pelo mesmo item.
3. **shouldSkipPrefetch()** ( `usePrefetchOnHover.ts:22-27` ) — respeita `navigator.connection.saveData` (usuários com data-saver ligado) e `effectiveType` `2g` / `slow-2g` . Prefetch em rede lenta pioraria a experiência.

Se `fn()` lança sincronamente, `firedRef` é resetada ( `usePrefetchOnHover.ts:59-61` ) para permitir nova tentativa. Rejeições de `Promise` não são tratadas — o chunk simplesmente não fica em cache; o click posterior fará o fetch normal.

O cleanup em `useEffect(() => cancel, [cancel])` ( `usePrefetchOnHover.ts:76` ) garante que o timer é limpo se o componente desmontar durante o debounce.

## Comportamento por tipo de evento

| Evento                                          | Método interno                         | Justificativa                                                                 |
|-------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------|
| <code>onmouseenter</code>                       | <code>schedule</code> (120ms debounce) | Evita disparo em movimento rápido do cursor atravessando a lista              |
| <code>onFocus</code>                            | <code>schedule</code> (120ms debounce) | Tab-navigation sequencial                                                     |
| <code>onTouchStart</code>                       | <code>scheduleImmediate</code>         | Toque é intenção clara; latência mobile é o gargalo, não podemos gastar 120ms |
| <code>onMouseLeave</code> / <code>onBlur</code> | <code>cancel</code>                    | Reverte antes do timer disparar                                               |

`scheduleImmediate` ( `usePrefetchOnHover.ts:65-74` ) faz o mesmo trabalho de `schedule` mas sem `setTimeout` — respeita as mesmas guardas ( `enabled` , `firedRef` , `shouldSkipPrefetch` ).

## Padrão de wiring por linha

Regra dos hooks: **não podemos** chamar `usePrefetchOnHover` dentro de um `.map()` . Solução: extrair um sub-componente de linha que chame o hook uma vez por render.

Exemplo em `src/pages/clients/ClientsPage.tsx:22-30` :

```
function ClientRow({ client, onClick }: { client: CrmCompany; onClick: () => void }) {
  const handlers = usePrefetchOnHover(prefetchClientDetailChunk);
  return <ClientCard client={client} onClick={onClick} prefetchHandlers={handlers} />;
}
```

O `ClientCard` ( `src/components/clients/ClientCard.tsx` ) foi ajustado para aceitar uma prop opcional `prefetchHandlers: PrefetchHandlersLike` que é espalhada no `<Card>` raiz. Assim o handler chega ao

DOM sem que o card precise conhecer o conceito de prefetch.

Mesmo padrão em `src/components/quotes/QuotesConfigurableList.tsx:55-60` , onde o wrapper `QuoteRow` recebe uma `prefetch` fn diferente por linha (prefetch do `QuoteViewPage` e do `QuoteBuilderPage` em paralelo).

## Prefetch do chunk lazy

A fn passada ao hook é uma seta que executa um `import()` dinâmico:

```
// src/pages/clients/ClientsPage.tsx
const prefetchClientDetailChunk = () => import('@pages/clients/ClientDetailPage');
```

Vantagens dessa forma:

- **Reuso automático de promise** — Vite e o React (via `React.lazy` ) memoizam `import()` por especificador. Quando o hover dispara o prefetch, o chunk vai para o cache HTTP e o registro interno do Vite; quando o `React.lazy` do detalhe é montado pelo click, ele reusa a mesma promise resolvida sem novo network round-trip.
  - **Zero código extra** — não precisamos manter um mapa de rotas prefetchadas nem coordenar com o `lazyWithRetry` .
  - **Falha silenciosa** — se o import falhar (offline, chunk quebrado), o hook engole a rejeição implicitamente e o click posterior tenta de novo pelo caminho normal, acionando o `lazyWithRetry` / `attemptChunkRecovery` .
-

# Apêndice

## Env vars e flags consolidadas

| Variável                          | Onde                                    | Default                    | Efeito                                           |
|-----------------------------------|-----------------------------------------|----------------------------|--------------------------------------------------|
| VITE_ENABLE_NAV_METRICS           | .env.production:11 ,<br>.env.example:73 | true em prod, false em dev | Liga/desliga coleta de métricas no build         |
| VITE_NAV_METRICS_SAMPLE_RATE      | .env.example:76                         | 0.1                        | Fração [0,1] de métricas enviadas ao Sentry      |
| VITE_SENTRY_DSN                   | .env.example:50                         | vazio                      | Sem DSN, captureMessage é no-op                  |
| VITE_SENTRY_ENVIRONMENT           | .env.example:54                         | import.meta.env.MODE       | Tag environment no Sentry                        |
| localStorage.nav_metrics_disabled | Runtime, por navegador                  | ausente                    | '1' desliga instrumentação mesmo com flag ligada |

## Comandos úteis

```
# Regerar snapshot após refactor intencional de chunking
npm run check:bundle-size:update

# Rodar o gate localmente (após npm run build)
npm run check:bundle-size

# Gerar report markdown localmente (após npm run build)
node scripts/bundle-size-report.mjs

# Rodar apenas os testes do kill switch
npx vitest run src/lib/telemetry/__tests__/navigationMetrics.test.ts

# Simular kill switch no DevTools do navegador
localStorage.setItem('nav_metrics_disabled', '1');
# Reabilitar
localStorage.removeItem('nav_metrics_disabled');
```

## Arquivos-fonte (path:line)

### Bundle:

- scripts/check-bundle-size.mjs:1 — gate bloqueante
- scripts/bundle-size-report.mjs:1 — report informativo
- bundle-size-baseline.json — snapshot canônico
- .github/workflows/bundle-size-report.yml:1 — workflow do PR-comment

- `.github/workflows/quality-gate.yml:100` — step do gate bloqueante
- `package.json:48-49` — scripts npm

### Métricas:

- `src/lib/telemetry/navigationMetrics.ts:1` — instrumentação
- `src/lib/telemetry/__tests__/navigationMetrics.test.ts:1` — testes do kill switch
- `src/lib/sentry.ts:209` — export de `captureMessage`
- `src/main.tsx:6-19` — bootstrap
- `src/components/common/RouteScrollReset.tsx:31` — hook de troca de rota
- `.env.production:11` — flag em produção
- `.env.example:73-79` — documentação de vars

### Prefetch:

- `src/hooks/common/usePrefetchOnHover.ts:37` — hook
- `src/pages/clients/ClientsPage.tsx:14-30` — wiring em Clientes
- `src/components/clients/ClientCard.tsx` — prop `prefetchHandlers`
- `src/components/quotes/QuotesConfigurableList.tsx:11-60` — wiring em Orçamentos

### Referências cruzadas com memória de projeto

- `mem://architecture/feature-flags-governance` — governança de feature flags.
- `mem://observability/structured-logging-and-correlation` — logger SSOT com `request_id`, complementar às métricas cobertas aqui.
- `mem://architecture/performance-virtualization-standards` — virtualização em listas > 100 items ( `@tanstack/react-virtual` ), complementar ao prefetch.
- `mem://infrastructure/chunk-recovery-system` — `attemptChunkRecovery` que atua como rede de segurança para falhas do prefetch/lazy.